Q65. What does sorted([(1,"b"),(2,"a"),(1,"a")], key=lambda x:(x[0],x[1])) return?

a) [(1,"a"),(1,"b"),(2,"a")] b) [(1,"b"),(1,"a"),(2,"a")] c) [(2,"a"),(1,"a"),(1,"b")] d) Error

Q66. What does list(enumerate(["a","b","c"], start=1)) return?

a) [(0,"a"),(1,"b"),(2,"c")] b) [(1,"a"),(2,"b"),(3,"c")] c) [1,2,3] d) ["a","b","c"]

Q67. What is the key difference between list.sort() and sorted(list)?

a) No difference b) sort() modifies in place, returns None; sorted() returns new list c) sorted() modifies in place; sort() returns new list d) sort() only works on numbers

Q68. What does [x for x in range(10) if x%3==0 if x%2==0] return?

a) [0,6] b) [0,3,6,9] c) [0,2,4,6,8] d) [6]

Q69. What does [1,2,3].index(5) raise?

a) None b) -1 c) ValueError d) IndexError

Q70. Which is more memory-efficient for large sequences?

a) List comprehension b) Generator expression c) Both use same memory d) Depends on element type

Q71. What does any([0, None, False, [], ""]) return?

a) True b) False c) None d) []

Q72. What is all([1, "hello", 3.14, True])?

a) False b) True c) 1 d) Error

Q73. What does [0]*3 produce?

a) [0,0,0] b) [0,3] c) 0 d) Error

Q74. What does list(reversed([1,2,3])) return?

a) [3,2,1] b) [1,2,3] c) a reversed object d) Error

Q75. When is a regular for loop preferable to a list comprehension?

a) Never - comprehensions always win b) When the body has multiple operations or side effects c) When data has >100 elements d) When using if conditions

TOPIC 4: DICTIONARIES & SETS (Q76–Q100)

Q76. What does d={"a":1}; d.get("b", 0) return?

a) None b) KeyError c) 0 d) {}

Q77. What happens after d={"x":1,"y":2}; d.update({"y":10,"z":3})?

a) d unchanged b) d={"x":1,"y":10,"z":3} c) d={"x":1,"y":2,"z":3} d) Error

Q78. What does d={}; d["key"] raise?

a) None b) ValueError c) KeyError d) IndexError

Q79. Which can be a dictionary key?

a) [1,2,3] b) {"a":1} c) (1,2,3) d) {1,2,3}

Q80. What does {k:v for k,v in [("a",1),("b",2)] if v>1} return?

a) {"a":1,"b":2} b) {"b":2} c) [("b",2)] d) {"b"}

Q81. What is len({"a","b","a","c","b"})?

a) 5 b) 3 c) 2 d) Error

Q82. What does {1,2,3} & {2,3,4} return?

a) {1,2,3,4} b) {2,3} c) {1,4} d) {1,2,3,2,3,4}

Q83. What is {1,2,3} - {2,3,4}?

a) {1} b) {4} c) {1,4} d) {}

Q84. What does {1,2,3} ^ {2,3,4} return?

a) {1,4} b) {2,3} c) {1,2,3,4} d) {}

Q85. What is the time complexity of set membership check vs list?

a) O(n) for both b) O(1) for set, O(n) for list c) O(n) for set, O(1) for list d) O(log n) for set, O(n) for list

Q86. What does dict.fromkeys(["a","b","c"], 0) return?

a) [("a",0),("b",0),("c",0)] b) {"a":0,"b":0,"c":0} c) {"a","b","c"} d) Error

Q87. What does d={"a":1,"b":2}; del d["a"]; print(d) output?

a) {"a":1,"b":2} b) {"b":2} c) {} d) Error

Q88. Are Python dicts ordered by insertion in Python 3.7+?

a) No, dicts are unordered b) Yes, insertion order is guaranteed c) Only with OrderedDict d) Depends on the Python version

Q89. What does set("hello") produce?

a) {"h","e","l","l","o"} b) {"h","e","l","o"} c) ["h","e","l","l","o"] d) "hello"

Q90. Difference between .discard() and .remove() for sets?

a) No difference b) discard() raises KeyError if absent c) remove() raises KeyError if absent; discard() silently ignores d) discard() returns new set

Q91. What does {**{"a":1}, **{"b":2}, **{"a":10}} return?

a) {"a":1,"b":2} b) {"a":10,"b":2} c) {"a":[1,10],"b":2} d) Error - duplicate key

Q92. What is the Pythonic way to build a frequency counter in a plain dict?

a) Check with if key in dict then increment b) Use .get(key,0) + 1 c) Use setdefault(key,0) then increment d) Both B and C are valid

Q93. What does frozenset({1,2,3}) | frozenset({3,4}) return?

a) Error - frozensets are immutable b) frozenset({1,2,3,4}) c) [1,2,3,4] d) {1,2,3,4}

Q94. What does d={"a":1,"b":2}; list(d.items()) return?

a) ["a","b"] b) [1,2] c) [("a",1),("b",2)] d) {"a":1,"b":2}

Q95. What does "a" in {"a":1, "b":2} check?

a) Whether "a" is in the values b) Whether "a" is in the keys c) Whether ("a",1) is in items d) Error

Q96. What does d.pop("key", None) do when "key" doesn't exist?

a) Raises KeyError b) Returns None c) Returns {} d) Does nothing (no return)

Q97. Which creates an empty dict, NOT an empty set?

a) set() b) {} c) frozenset() d) dict.fromkeys([])

Q98. What does max({"a":3, "b":1, "c":2}) return?

a) 3 b) "c" c) "a" d) Error

Q99. What is the average time complexity of dict insertion?

a) O(n) b) O(log n) c) O(1) amortized d) O(n²)

Q100. What does dict(zip("abc", [1,2,3])) produce?

a) {"a":1,"b":2,"c":3} b) [("a",1),("b",2),("c",3)] c) Error d) {"abc":[1,2,3]}

TOPIC 5: CONTROL FLOW & LOOPS (Q101–Q125)

Q101. What does the ternary expression "pos" if x>0 else "neg" evaluate to for x=10?

a) "neg" b) "pos" c) True d) None

Q102. What does list(range(5,0,-2)) produce?

a) [5,3,1] b) [5,4,3,2,1] c) [5,3,1,-1] d) [0,2,4]

Q103. What prints when continue fires at i==1 in range(3)?

a) 0 1 2 b) 0 2 c) 1 2 d) 0

68	**A**	Multiple if clauses are AND'd. Divisible by both 3 and 2 = divisible by 6: 0, 6.
69	**C**	index() raises ValueError when the value is not found. Use "5 in lst" to check first.
70	**B**	Generator expressions are lazy - they yield one item at a time without storing all in memory.
71	**B**	any() returns True if AT LEAST ONE element is truthy. All elements here are falsy.
72	**B**	all() returns True if EVERY element is truthy. 1, non-empty strings, non-zero floats, and True are all truthy.
73	**A**	List multiplication repeats the list. [0]*3 = [0,0,0]. Warning: for mutable objects like lists, use list comprehension instead.
74	**A**	reversed() returns an iterator. list() materialises it: [3,2,1].
75	**B**	Comprehensions are for building lists. If you're printing, logging, or updating multiple variables, use a for loop for clarity.
76	**C**	get() returns the default (0 here) when the key is absent. No exception is raised.
77	**B**	update() merges dicts, overwriting existing keys with new values and adding new keys.
78	**C**	Accessing a missing dict key raises KeyError. Use .get() or check with "key in d" first.
79	**C**	Dict keys must be hashable. Tuples (with hashable elements) are hashable. Lists, dicts, and sets are not.

80	**B**	Dict comprehension with filter: only pairs where v>1 are included. Only ("b",2) qualifies.
81	**B**	Sets automatically deduplicate. {"a","b","a","c","b"} = {"a","b","c"}, length 3.
82	**B**	& is intersection: elements present in BOTH sets. 2 and 3 appear in both.
83	**A**	Set difference: elements in the first but NOT in the second. Only 1 qualifies.
84	**A**	^ is symmetric difference: elements in either set but NOT in both. 1 and 4 are unique to each set.
85	**B**	Sets use hash tables: O(1) average lookup. Lists require linear scan: O(n).
86	**B**	fromkeys() creates a dict with each key from the iterable mapped to the default value.
87	**B**	del removes the key-value pair from the dict. The key "a" and its value 1 are gone.
88	**B**	Since Python 3.7, dict preserves insertion order as a language guarantee (not just CPython detail).
89	**B**	Sets deduplicate. "hello" has two l's: set gives {"h","e","l","o"} - 4 elements.
90	**C**	remove() raises KeyError for missing elements. discard() is the safe version - no error if absent.
91	**B**	Dict unpacking merges dicts. Later values overwrite earlier ones for duplicate keys.
92	**D**	Both are Pythonic. For production code, collections.Counter is even cleaner.

93	**B**	Frozensets support all set operations but return frozenset (for union/intersection) since they are immutable.
94	**C**	.items() returns dict_items view of (key,value) tuples. list() materialises it.
95	**B**	in on a dict checks keys only. Use ".values()" or ".items()" to check values/pairs.
96	**B**	pop() with a default argument returns the default instead of raising KeyError.
97	**B**	{} creates an empty DICT. To create an empty set, use set() - not {}.disabled
98	**C**	Iterating over a dict yields keys. max("a","b","c") lexicographically = "c".
99	**C**	Hash tables have O(1) average insertion. Worst case (hash collisions) is O(n) but rare.
100	**A**	zip("abc",[1,2,3]) = [("a",1),("b",2),("c",3)]. dict() converts list of pairs to dict.
101	**B**	Python ternary: value_if_true if condition else value_if_false. x=10>0 is True so "pos".
102	**A**	Start=5, stop=0 (exclusive), step=-2: 5, 3, 1. Next would be -1 which is <= 0, so stop.
103	**B**	continue skips the rest of the loop body and moves to the next iteration. i=1 is skipped.
104	**B**	The else clause runs only if no break was encountered. It's an "exhausted normally" signal.
105	**B**	break also prevents the else clause